

Machine Learning Theory

HW/SW Codesign - Week 4

Today's Topics

- Terminology
 - Supervised vs unsupervised vs reinforcement learning
 - Classification vs regression
- Linear regression
 - Loss and cost functions
 - Gradient descent
 - Learning rate
 - Batch size
- Overfitting and regularization
 - Polynomial regression
 - Generalization and overfitting
 - Regularization
 - Variance and bias
- Logistic regression
 - Binary classification
 - Softmax regression
- Model evaluation
 - Validation vs test set
 - Accuracy, precision, recall and F1 score
 - Precision/recall trade-off
- Classical models vs deep learning

Supervised vs Unsupervised vs Reinforcement Learning

- Supervised learning
 - Examples with labels
 - Applications: Recognition, prediction
 - What we do
- Unsupervised learning
 - Data points without labels
 - Applications: Clustering, anomaly detection, dimensionality reduction
- Reinforcement learning
 - Interaction with environment
 - Applications: Robotics, games

Classification vs Regression

- Classification
 - Predicts a class
 - Binary classification
 - Two classes, negative and positive
 - Example: Banana image recognition
 - Output: Class index or probability
 - Multi-class classification
 - Three or more classes; one may be negative
 - Example: Keyword spotter
 - Output: Class index or vector of probabilities

Classification vs Regression

- Regression
 - Predicts a number
 - Example: Life satisfaction, House prices
 - Example: Step length from accelerometer
 - Or a vector or matrix of numbers
 - Example: Density map for people counting
 - Example: Spectrum for modified audio in hearing aid

Linear Regression

The foundational ML model

Machine Learning Model

A **machine learning model** is a generic mathematical function governed by **parameters** that can be adjusted to train the model for a specific task.

$$\hat{\mathbf{y}} = f(\mathbf{x}, \theta)$$

$\hat{\mathbf{y}} \in R^k$ is the prediction,
 $\mathbf{x} \in R^n$ is the feature vector,
 $\theta \in R^p$ are the parameters.

Linear Regression

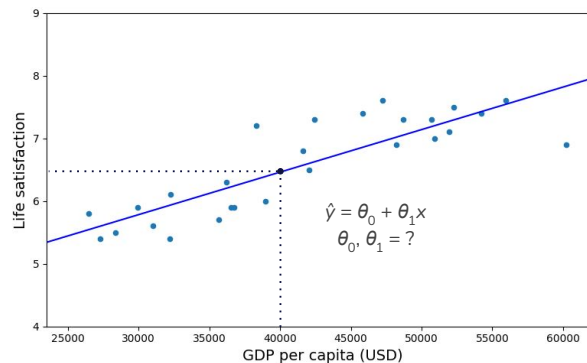
Linear regression model with one variable:

$$f(x) = \theta_0 + \theta_1 x$$

$x \in R$ is the feature,
 $\theta \in R^2$ are the parameters.

Linear Regression

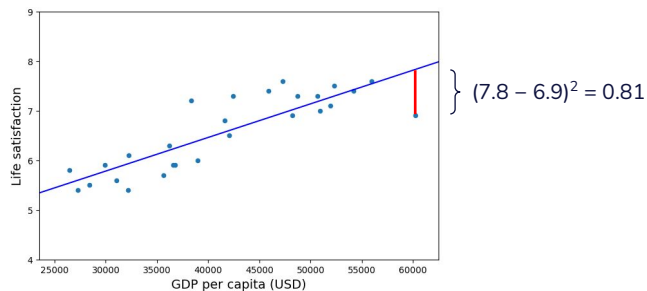
- Linear regression finds θ_0 and θ_1 such that $y = \theta_0 + \theta_1 x$ fits the given data
- $\hat{y} = \theta_0 + \theta_1 x$ can be used to predict life satisfaction y given GDP per capita x



Linear Regression: Loss Function

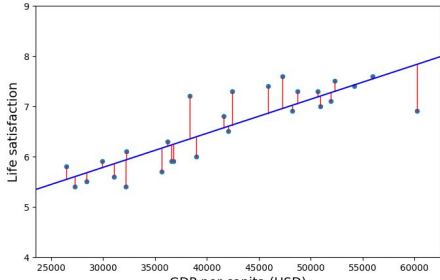
- **Loss function:** Error function for a single example (x, y)
- **Squared error** for linear regression:

$$SE = (f(x, \theta) - y)^2 = (\theta_0 + \theta_1 x - y)^2$$



Linear Regression: Cost Function

- **Cost function:** Error function on an entire dataset $(\mathbf{X}, \mathbf{y})$
- **Mean squared error (MSE)** for linear regression:

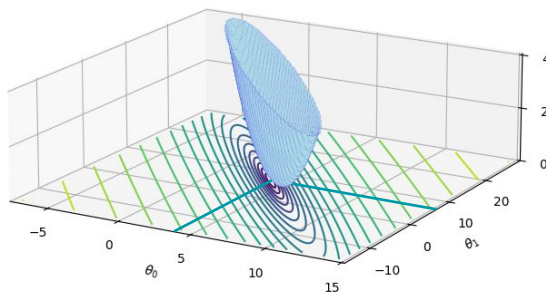
$$\text{MSE}(\theta) = \frac{1}{m} \sum_{i=1}^m (f(x_i, \theta) - y_i)^2 = \frac{1}{m} \sum_{i=1}^m (\underbrace{\theta_0 + \theta_1 x_i}_{\text{Prediction}} - \underbrace{y_i}_{\text{Label}})^2$$


The scatter plot shows 'Life satisfaction' on the y-axis (ranging from 4 to 9) and 'GDP per capita (USD)' on the x-axis (ranging from 25,000 to 60,000). Blue dots represent data points, and a solid blue line represents the linear regression fit. Red vertical lines connect each data point to the regression line, representing the residuals.

Linear Regression: Minimizing the Cost Function

- Fitting $\theta_0 + \theta_1 x$ to dataset $(\mathbf{X}, \mathbf{y})$ amounts to minimizing the cost function:

$$\text{MSE}(\theta) = \frac{1}{m} \sum_{i=1}^m (\theta_0 + \theta_1 x_i - y_i)^2$$



Linear Regression: Minimizing the Cost Function

- Fitting $\theta_0 + \theta_1 x$ to dataset $(\mathbf{X}, \mathbf{y})$ amounts to minimizing the cost function:

$$\text{MSE}(\theta) = \frac{1}{m} \sum_{i=1}^m (\theta_0 + \theta_1 x_i - y_i)^2$$

- Closed-form solutions:
 - Normal equation: $\hat{\theta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$
 - Pseudoinverse: $\hat{\theta} = \mathbf{X}^+ \mathbf{y}$
 - Only work for linear regression
- Numeric solutions:
 - Gradient descent: $\theta' = \theta - \eta \nabla \text{MSE}(\theta)$
 - Works for many models

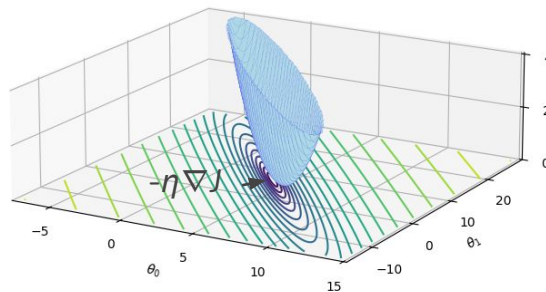
Gradients

- The **gradient** ∇J of a scalar multivariate function $J(\theta)$ is a vector of all its partial derivatives:
- The **partial derivative** with respect to θ_1 is the rate at which J changes when only θ_1 varies

$$\nabla J = \left(\frac{\partial J}{\partial \theta_1}, \frac{\partial J}{\partial \theta_2}, \dots, \frac{\partial J}{\partial \theta_n} \right)$$

Gradient Descent

- The gradient at a point θ points in the direction of steepest ascent
- If we go in the opposite direction, the value of f will decrease
- $\theta' = \theta - \eta \nabla J(\theta)$, where η is the **learning rate**
- η is a **hyperparameter** that we can choose



Linear Regression: Gradient Descent

Let's compute the partial derivatives of $\text{MSE}(\theta)$:

- $$\text{MSE}(\theta) = \frac{1}{m} \sum_{i=1}^m (\theta_0 + \theta_1 x_i - y_i)^2$$
- $$\frac{\partial}{\partial \theta_0} \text{MSE}(\theta) = \frac{2}{m} \sum_{i=1}^m (\theta_0 + \theta_1 x_i - y_i)$$
- $$\frac{\partial}{\partial \theta_1} \text{MSE}(\theta) = \frac{2}{m} \sum_{i=1}^m (\theta_0 + \theta_1 x_i - y_i) x_i$$

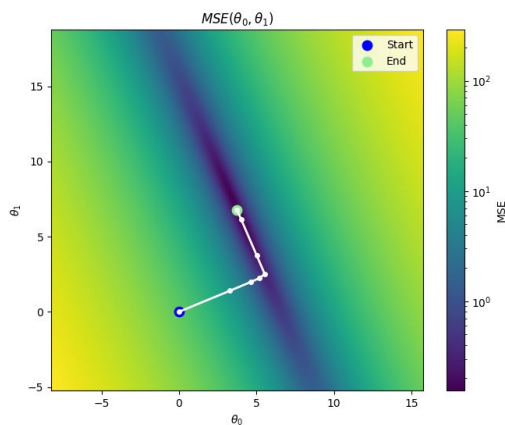
Linear Regression: Gradient Descent

- We now have a formula for $\nabla \text{MSE}(\theta)$
- We will perform gradient descent:

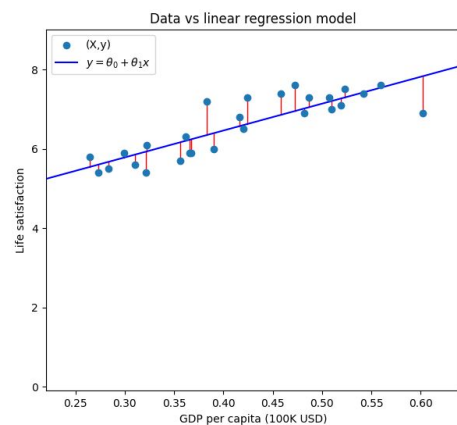
$$\theta' = \theta - \eta \nabla \text{MSE}(\theta)$$

- Every iteration is called an **epoch**
- We choose learning rate $\eta = 0.25$

Linear Regression: Gradient Descent



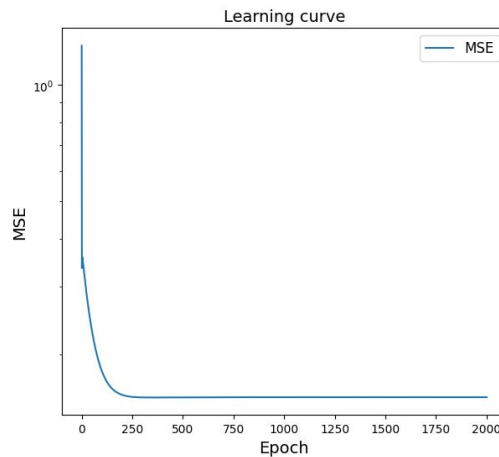
Epoch: 2000
 $\theta_0 = 3.75, \theta_1 = 6.78$



$y = 3.75 + 6.78x$
MSE = 0.1539

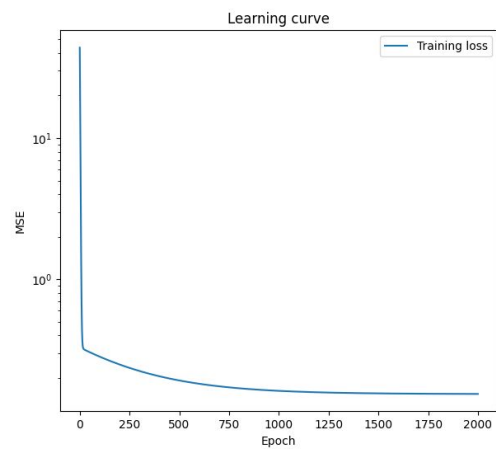
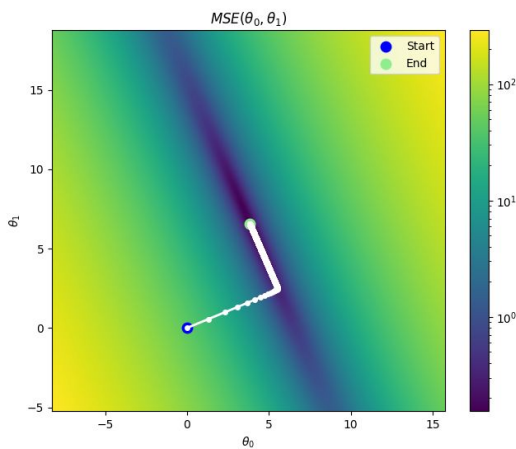
Linear Regression: Learning Curve

- The **learning curve** (aka **loss plot**) shows cost over epochs



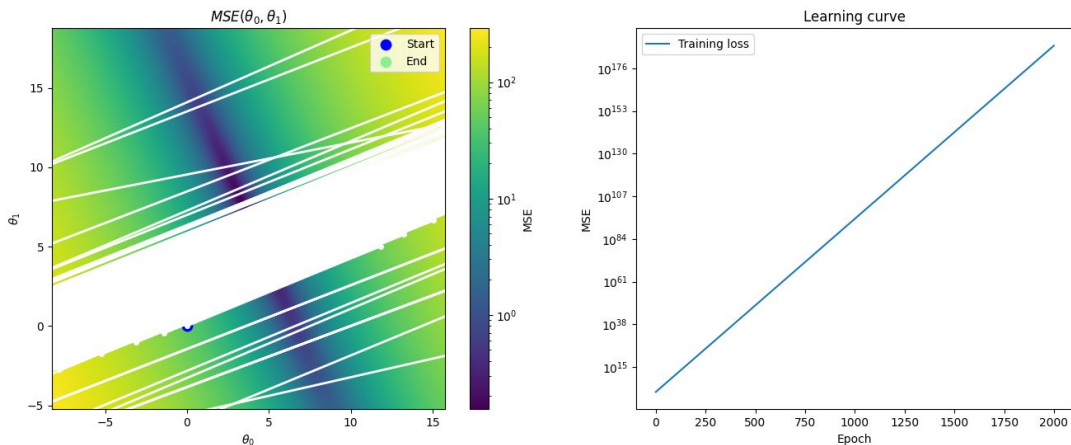
Linear Regression: Learning Rate

- Learning rate $\eta = 0.1$: Too low, doesn't reach optimum



Linear Regression: Learning Rate

- Learning rate $\eta = 0.9$: Too high, diverges



Mini-Batches

- Large datasets can be split into **mini-batches**
- **Batch size** determines the size of each batch
- Batch size is another hyperparameter that we choose
- Each epoch loops over the mini-batches
 - Example: 3162 examples, batch size=64
 - $\Rightarrow$ 50 mini-batches per epoch
- Advantages:
 - Saves memory and fits hardware accelerator
 - Neural networks: Escapes local minima $\Rightarrow$ Better model

Linear Regression: Multiple Variables

Linear regression model with n variables:

$$f(\mathbf{x}, \theta) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$$

$\mathbf{x} \in R^n$ is the feature vector,

$\boldsymbol{\theta} \in R^{n+1}$ is the parameter vector.

Linear Regression: Matrix Form

Linear regression model with n variables in **matrix form**:

$$f(\mathbf{x}, \theta) = \theta^T \mathbf{x}$$

$\mathbf{x} \in R^{n+1}$ is the feature matrix with $x_0 = 1$,

$\boldsymbol{\theta} \in R^{n+1}$ is the parameter matrix.

Linear Regression: Cost Function in Matrix Form

Cost function on a dataset $(\mathbf{X}, \mathbf{y})$:

$$\text{MSE}(\theta) = \frac{1}{m} \sum_{i=1}^m (\theta^T \mathbf{x}_i - y_i)^2$$

$\theta \in R^{n+1}$ is the parameter matrix

$\mathbf{x}_i \in R^{n+1}$ is the feature matrix for example i with $x_0 = 1$,

y_i is the label for example i .

Overfitting and Regularization

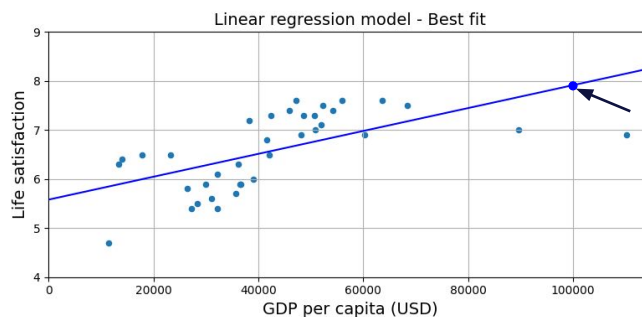
When optimum is not optimal

Overfitting

- **Overfitting** is when the model learns the training data too well, but fails to **generalize** to new, unseen data.
- Overfitting happens when:
 - Your model is complex, and
 - You don't have enough data
- Can be checked with the validation set

Underfitting

- Linear regression is extremely simple
 - Doesn't fit more complex relationships
- Example:
 - $f(100000 \text{ USD}) \approx 7.9$



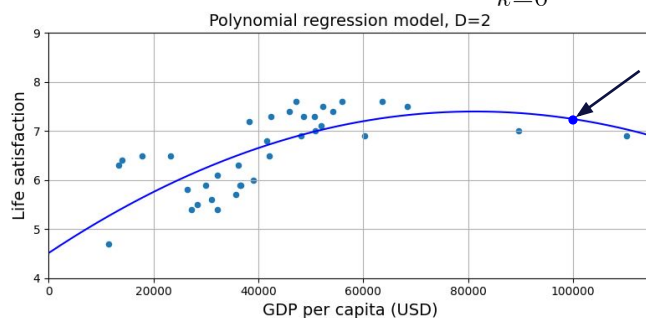
Appropriate Fit

- Polynomial regression degree 2: Generalizes well to new unseen countries

- Example:

- $f(100000 \text{ USD}) \approx 7.2$

$$f(x) = \sum_{k=0}^D (\theta_k x^k)$$



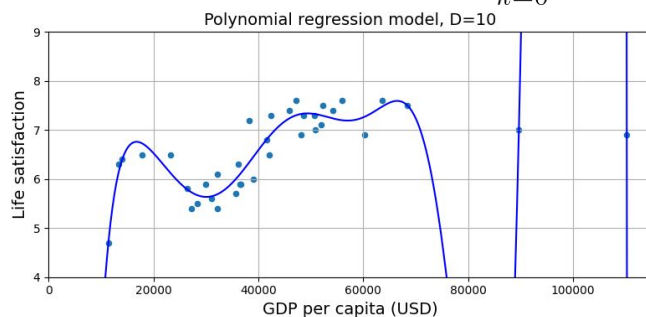
Overfitting

- Polynomial regression degree 10: Doesn't generalize to new unseen countries

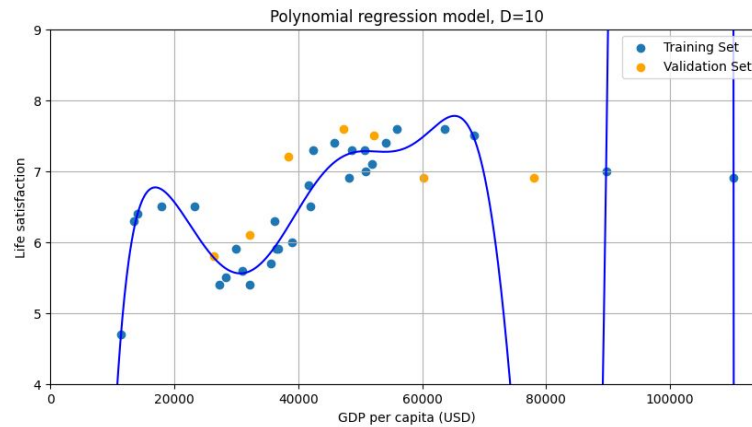
- Example:

- $f(100000 \text{ USD}) \approx 135$

$$f(x) = \sum_{k=0}^D (\theta_k x^k)$$



Overfitting: Assessment with Validation Set



MSE on training set: **0.0506**

MSE on validation set: **8.59**

Overfitting: Prevention

- Overfitting can be prevented with:
 - Less complex model (fewer parameters)
 - More data (including data augmentation)
 - Regularization

Regularization

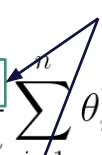
- Regularization constrains the model to prevent it from overfitting
- Techniques:
 - Ridge regression
 - Lasso regression
 - Early stopping

Regularization: Ridge and Lasso Regression

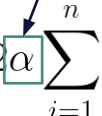
- **Ridge regression** (L2 regularization):

$$J(\theta) = \text{MSE}(\theta) + \frac{\alpha}{m} \sum_{i=1}^n \theta_i^2$$

regularization strength
(hyperparameter)

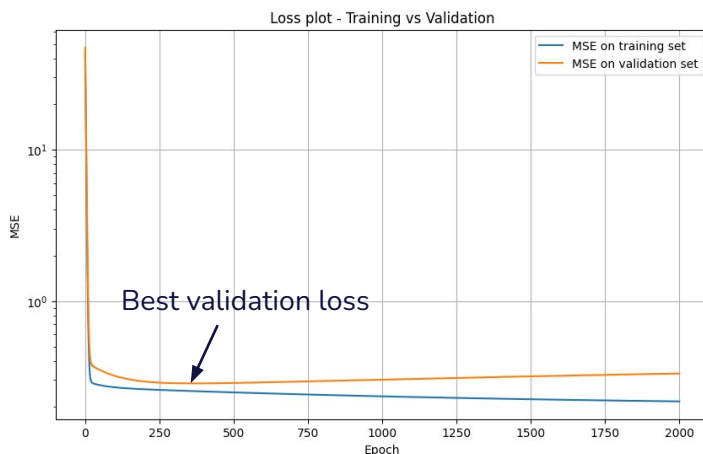


- **Lasso regression** (L1 regularization):

$$J(\theta) = \text{MSE}(\theta) + 2\alpha \sum_{i=1}^n |\theta_i|$$


Regularization: Early Stopping

- Stop gradient descent when validation loss starts increasing



Variance and Bias

- The **generalization error** is the prediction error on new, unseen inputs
 - Example: Difference between $f(100000 \text{ USD})$ and the true life satisfaction
- Generalization error is a sum of **variance** and **bias**
 - Variance: Generalization error due to overfitting
 - Example: $f(100000 \text{ USD}) \approx 135$ with 10-degree polynomial
 - Bias: Generalization error due to underfitting
 - Example: $f(100000 \text{ USD}) \approx 7.9$ with linear model
- Trade-off:
 - More parameters $\Rightarrow$ More variance, less bias
 - Fewer parameters, more regularization $\Rightarrow$ Less variance, more bias

Logistic Regression

How to turn regression into classification

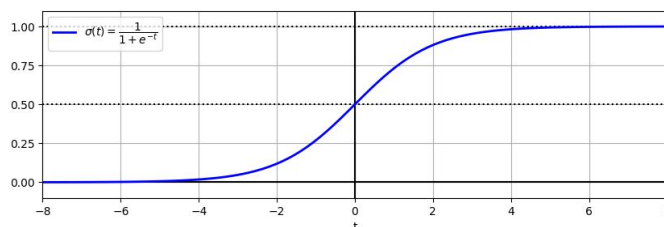
Logistic Regression

Logistic regression model for **binary classification**:

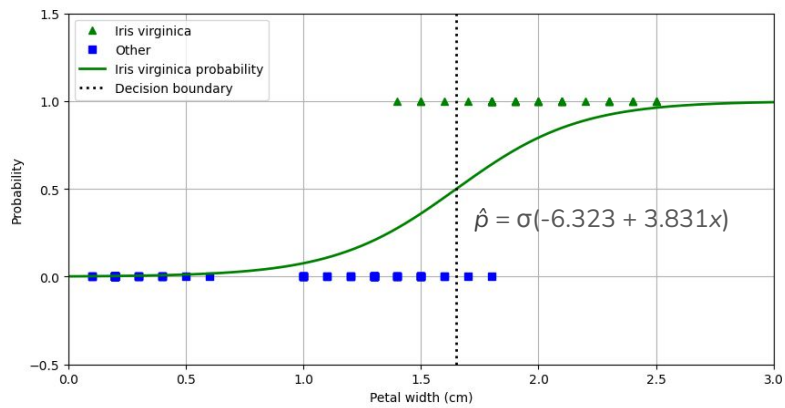
$$\hat{p} = \sigma(\theta^T \mathbf{x})$$

where $\sigma(t)$ is the **sigmoid function**, which maps any real value smoothly to $[0, 1]$:

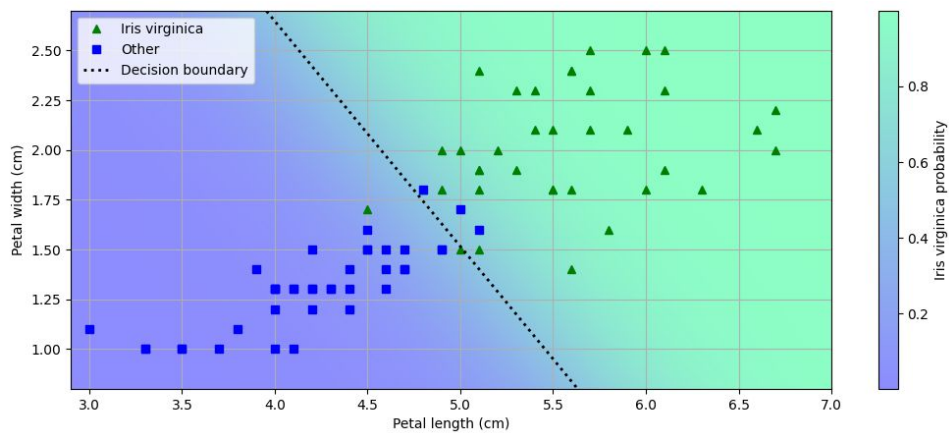
$$\sigma(t) = \frac{1}{1 + e^{-t}}$$



Logistic Regression: One Variable



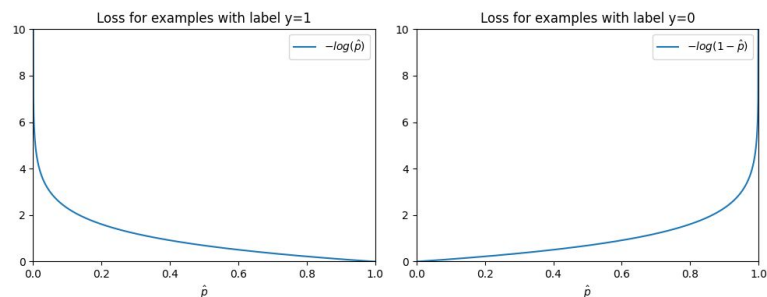
Logistic Regression: Two Variables



Logistic Regression: Loss Function

- **Log loss** for logistic regression $\hat{p} = \sigma(\theta^T \mathbf{x})$:

$$L(\theta) = \begin{cases} -\log(\hat{p}) & \text{if } y = 1 \\ -\log(1 - \hat{p}) & \text{if } y = 0 \end{cases}$$



Logistic Regression: Cost Function

Cost function for logistic regression on dataset $(\mathbf{X}, \mathbf{y})$:

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m (y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i)),$$
$$\hat{p}_i = \sigma(\theta^T \mathbf{x}_i)$$

Softmax Regression

Softmax regression model for **multi-class classification** with K classes:

$$(\hat{p}_1, \hat{p}_2, \dots, \hat{p}_K) = \text{softmax}((\theta_1^T \mathbf{x}, \theta_2^T \mathbf{x}, \dots, \theta_K^T \mathbf{x}))$$

where:

$$\text{softmax}(\mathbf{s})_k = \frac{e^{s_k}}{\sum_{j=1}^K e^{s_j}}$$

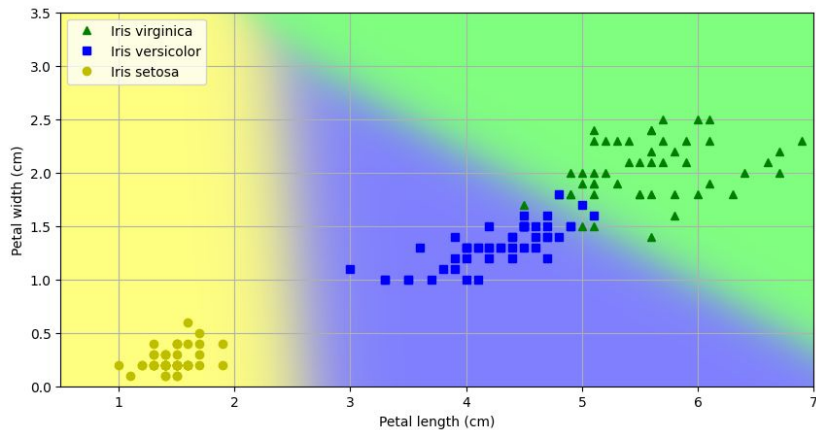
with the decision function:

$$\hat{y}(\hat{\mathbf{p}}) = \underset{i}{\operatorname{argmax}} p_i$$

Softmax Regression

- Logistic regression for multi-class classification
- Use one linear regressor for each class k : $s_k = \boldsymbol{\theta}_k^T \mathbf{x}$
- Example:
 - $\boldsymbol{\theta}_1 = [20.94, 5.191, -2.692]^T$ (Parameters for Iris Setosa)
 - $\boldsymbol{\theta}_2 = [6.485, 0.3529, -2.318]^T$ (Parameters for Iris Versicolor)
 - $\boldsymbol{\theta}_3 = [-27.43, 4.857, 5.01]^T$ (Parameters for Iris Virginica)
 - $\mathbf{x} = [1, 5.1, 1.8]^T$ (1, petal width, petal length)
 - $s_1 = \boldsymbol{\theta}_1^T \mathbf{x} = -10.37$ (Score for Iris Setosa)
 - $s_2 = \boldsymbol{\theta}_2^T \mathbf{x} = 4.113$ (Score for Iris Versicolor)
 - $s_3 = \boldsymbol{\theta}_3^T \mathbf{x} = 6.257$ (Score for Iris Virginica)
 - $\text{softmax}([-10.37, 4.113, 6.257]) = (0.000, 0.105, 0.895)$

Softmax Regression



Softmax Regression: Loss Function

- **Cross-entropy loss** for softmax regression $\hat{p}_k = \text{softmax}((\theta_1^T \mathbf{x}, \dots, \theta_K^T \mathbf{x}))_k$:

$$L((\theta_1, \dots, \theta_K)) = - \sum_{k=1}^K y_k \log(\hat{p}_k)$$

- The label $\mathbf{y}$ is a **one-hot vector** of length K:
 - $y_k = 1$ if k is the true class, 0 otherwise
- Example:
 - $\mathbf{y} = (0, 0, 1)$
 - $\hat{\mathbf{p}} = (0.08, 0.17, \underline{0.75})$
 - $\Rightarrow L = -\log(0.75) \approx 0.29$

Softmax Regression: Cost Function

- Cost function for softmax regression on dataset $(\mathbf{X}, \mathbf{y})$:

$$J((\theta_1, \dots, \theta_K)) = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K y_{i,k} \log(\hat{p}_{i,k}),$$
$$\hat{p}_{i,k} = \text{softmax}((\theta_1^T \mathbf{x}_i, \theta_2^T \mathbf{x}_i, \dots, \theta_K^T \mathbf{x}_i))_k$$

Model Evaluation

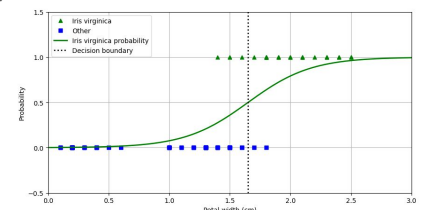
How to see if the model works

Validation vs Test Set

- Validation set:
 - Used during development to compare different models
 - Model will fit to validation set via development choices
 - $\Rightarrow$ Unreliable metrics after some development
- Test set:
 - Used once, in the end
 - $\Rightarrow$ Reliable metrics
 - Further development based on test set $\Rightarrow$ **test set contamination**
 - If you need to develop more, you must collect a new test set!

Classification Model Metrics

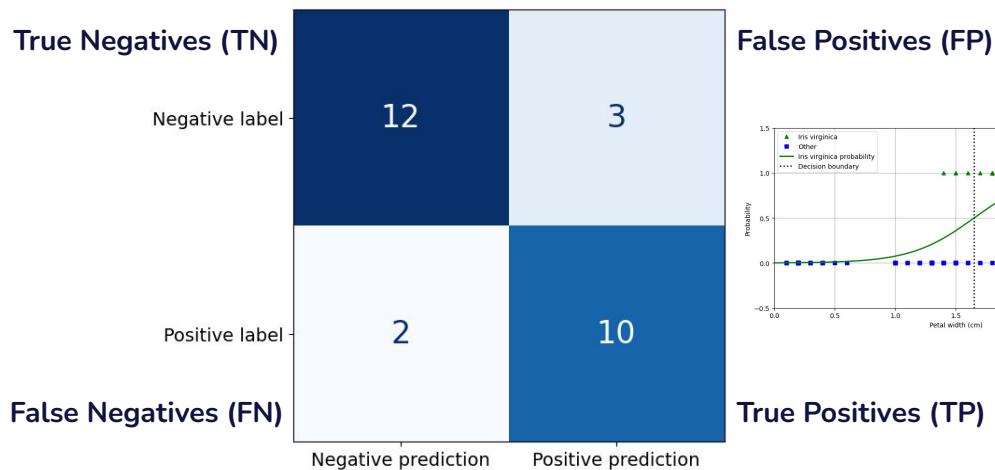
- **Accuracy:**
 - Ratio of correct predictions out of all examples
 - Example: $22 / 27 = 81.5\%$
- **Precision:**
 - Ratio of correct predictions out of all positive predictions
 - Example: $9 / 11 = 81.8\%$
- **Recall:**
 - Ratio of correct predictions out of all examples with positive labels
 - Example: $9 / 12 = 75\%$
- **F1 Score:**
 - Harmonic mean of precision and recall
 - Example: $2 * 0.818 * 0.75 / (0.818 + 0.75) = 77.8\%$



Classification Model Metrics

- $\text{accuracy} = \frac{\text{correct predictions}}{\text{all examples}}$
- $\text{precision} = \frac{\text{correct predictions}}{\text{positive predictions}}$
- $\text{recall} = \frac{\text{correct predictions}}{\text{positive examples}}$
- $F_1 = \frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$

Confusion Matrix

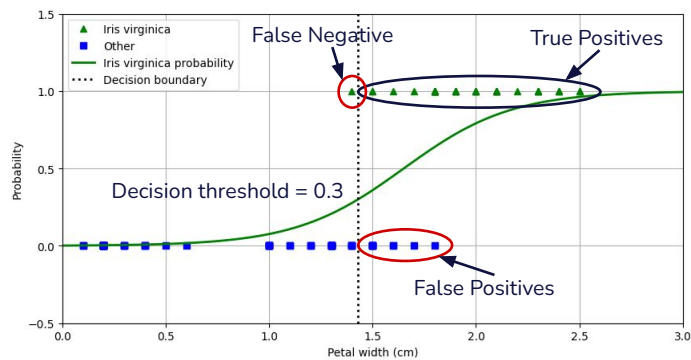


Classification Model Metrics from Confusion Matrix

- $\text{accuracy} = \frac{TN + TP}{TN + FP + FN + TP}$
- $\text{precision} = \frac{TP}{TP + FP}$
- $\text{recall} = \frac{TP}{TP + FN}$
- $F_1 = \frac{2TP}{2TP + FP + FN}$

Negative label	TN: 12	FP: 3
Positive label	FN: 2	TP: 10
	Negative prediction	Positive prediction

Precision/Recall Trade-Off

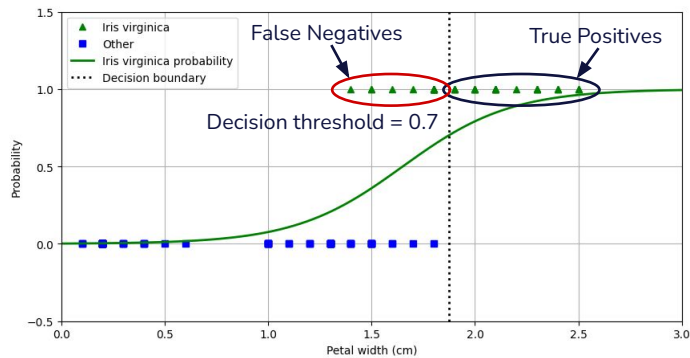


TP = 11; FN = 1; FP = 4

⇒ $\text{precision} = TP / (TP + FP) = 11 / 15 \approx 73.3\%$

⇒ $\text{recall} = TP / (TP + FN) = 11 / 12 \approx 91.2\%$

Precision/Recall Trade-Off



TP = 7; FN = 5; FP = 0

⇒ precision = $TP / (TP + FP) = 7 / 7 = 100\%$

⇒ recall = $TP / (TP + FN) = 7 / 12 = 58.3\%$

Precision/Recall Trade-Off

Examples:

- Shoplifter detection in surveillance
 - Prefer **high precision** (avoid accusing innocent customers)
 - ⇒ Higher threshold
- Cancer detection in CT scans:
 - Prefer **high recall** (better safe than sorry)
 - ⇒ Lower threshold
- Hövding inflatable bike helmet:
 - Prefer **higher recall** (better safe than sorry)
 - ... but **high precision** is also critical, so it doesn't go off all the time
 - ⇒ Balanced threshold

Classical Models

What you can use instead of deep learning

Classical Models vs Deep Learning

- Classical Models
 - Useful when you don't have much data
 - Requires small feature vectors $\Rightarrow$ more feature engineering
 - Common classical models:
 - Decision trees
 - Random forest
 - Support vector machine (SVM)

Classical Models vs Deep Learning

- Deep Learning
 - Neural networks with multiple layers
 - Can learn complex patterns in high-dimensional data
 - Requires more data